

LAPORAN TUGAS
ALJABAR LINEAR (PRAKTIKUM)
MODUL 2: OPERASI VEKTOR, NORMA, DAN JARAK

Laporan Ini Disusun Untuk Memenuhi Tugas Kuliah
Mata Kuliah : Aljabar Linear (Praktikum)



Disusun Oleh :
Nama : Ghaisan Khoirul Badruzaman
NIM : 251524048
Kelas : 2B-D4

PROGRAM STUDI D4 TEKNIK INFORMATIKA
JURUSAN TEKNIK KOMPUTER DAN INFORMATIKA
POLITEKNIK NEGERI BANDUNG
2026

DAFTAR ISI

BAB I PENDAHULUAN	3
1.1 Latar Belakang	3
1.2 Tujuan	3
1.3 Lingkungan dan Struktur Proyek	3
BAB II TASK I: NORMA-p DAN SIFATNYA	5
2.1 Deskripsi Program	5
2.2 Kode Program	5
2.3 Penjelasan Proses	5
2.4 Output Program	5
2.5 Lesson Learnt	6
BAB III TASK II: JARAK, KESAMAAN KOSINUS, DAN PROYEKSI	7
3.1 Deskripsi Program	7
3.2 Kode Program	7
3.3 Penjelasan Proses	7
3.4 Output Program	7
3.5 Lesson Learnt	8
BAB IV TASK III: STUDI KASUS KEMIRIPAN DOKUMEN	9
4.1 Deskripsi Program	9
4.2 Kode Program	9
4.3 Penjelasan Proses	9
4.4 Output Program	9
4.5 Analisis dan Lesson Learnt	10
BAB V TUGAS DAN LATIHAN	12
5.1 Latihan 1: Hubungan Norma	12
5.2 Latihan 2: Proyeksi dan Sudut	12
5.3 Tugas Rumah: Jarak Minkowski	12
5.4 Tantangan: Cauchy-Schwarz Menjadi Persamaan	13
BAB VI PENGUJIAN OTOMATIS DAN KUALITAS KODE	14
6.1 Unit Test dan Property Test	14
6.2 Hasil Pengujian	14
6.3 Kualitas Kode	15
BAB VII KESIMPULAN	16

BAB I PENDAHULUAN

1.1 Latar Belakang

Modul 1 berhenti di operasi dasar vektor: penjumlahan, dot product, dan norma Euclidean. Modul 2 melanjutkannya ke pertanyaan yang lebih praktis, yaitu seberapa jauh dua vektor dan seberapa mirip arahnya. Norma, jarak, kesamaan kosinus, dan proyeksi ortogonal adalah bahan dasar pencarian kemiripan, pembelajaran mesin, dan pengolahan sinyal. Bedanya dengan modul sebelumnya, sifat matematisnya tidak cukup dicek dengan satu dua contoh, jadi modul ini memakai property-based test yang mencoba ratusan vektor acak sekaligus.

1.2 Tujuan

Sesuai capaian pembelajaran Modul 2, praktikum ini bertujuan untuk:

1. Mengimplementasikan norma- p ($p = 1, 2$, dan tak hingga) dan jarak Minkowski secara manual maupun dengan nalgebra, lalu memverifikasi sifat norma: positif, homogen, dan ketaksamaan segitiga.
2. Menghitung kesamaan kosinus dan proyeksi ortogonal, lalu memaknainya secara geometris dan pada kasus kemiripan dokumen.
3. Merancang property-based test untuk ketaksamaan Cauchy-Schwarz dan ketaksamaan segitiga.

1.3 Lingkungan dan Struktur Proyek

Toolchain sama dengan Modul 1: rustc 1.98.1, cargo 1.98.1, clippy 0.1.98, dan nalgebra 0.35.0. Tambahan barunya proptest 1.11.0. Modul menaruh proptest di `[dependencies]`, tetapi di proyek ini proptest dipasang sebagai `[dev-dependencies]` karena hanya dipakai saat cargo test, sehingga tidak ikut dikompilasi ke program utama. Gambar 1.1 memperlihatkan versi toolchain, pohon dependensi, dan hasil clippy yang bersih.

```
praktikum-al — ghaisan@Ghaisans-MacBook-Pro — ../praktikum-al — -zsh — 100x16

> rustc --version && cargo --version && cargo clippy --version
rustc 1.98.1 (48a229cea 2026-09-01)
cargo 1.98.1 (797e8a9bc 2026-08-05)
clippy 0.1.98 (48a229cea 2026-09-01)
> cargo tree --depth 1
praktikum-al v0.1.0 (/Users/ghaisan/Documents/Kuliah/aljabar-linear/2026-09-14-modul-2/praktikum-al)
└─ nalgebra v0.35.0
[dev-dependencies]
└─ proptest v1.11.0
> cargo clippy --all-targets -- -D warnings
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.08s

~/Documents/Kuliah/aljabar-linear/2026-09-14-modul-2/praktikum-al on main ?1 at 09:05:49
>
```

Gambar 1.1: Versi toolchain, pohon dependensi, dan clippy tanpa warning

Kode dipecah per task supaya mudah dibaca dan diuji terpisah. Tiap berkas punya modul `tests` sendiri.

```
praktikum-al/
├─ Cargo.toml
└─ src/
    ├─ main.rs      pemilih task lewat argumen, fungsi bantu fmt_vec
    ├─ task1.rs     norma-p manual, perbandingan nalgebra, property test
    ├─ task2.rs     jarak, kesamaan kosinus, proyeksi, normalisasi
    ├─ task3.rs     studi kasus kemiripan dokumen
    └─ latihan.rs    latihan 1 dan 2, tugas rumah Minkowski, tantangan
```

Program dijalankan dengan `cargo run -- 1` sampai `cargo run -- 4` untuk satu bagian saja, atau `cargo run` tanpa argumen untuk semuanya. Seluruh pengujian dijalankan dengan `cargo test`.

BAB II TASK I: NORMA-p DAN SIFATNYA

2.1 Deskripsi Program

Task I membuat fungsi norma-p generik untuk slice, mengujinya pada $v = [3, -4]$, lalu membandingkan hasilnya dengan `norm()` dan `lp_norm()` milik nalgebra. Norma-p didefinisikan sebagai $\|v\|_p = (\text{jumlah } |v_i|^p)^{(1/p)}$. Untuk p tak hingga rumus itu diganti nilai mutlak komponen terbesar. Setelah itu tiga sifat norma diuji dengan proptest pada 200 vektor acak: homogenitas, ketaksamaan segitiga, dan Cauchy-Schwarz.

2.2 Kode Program

```
pub fn norma_p(v: &[f64], p: f64) -> f64 {
    if p.is_infinite() {
        v.iter().map(|x| x.abs()).fold(f64::NEG_INFINITY, f64::max)
    } else {
        v.iter().map(|x| x.abs().powf(p)).sum::<f64>().powf(1.0 / p)
    }
}
```

Pengujian sifat memakai toleransi relatif, bukan epsilon mutlak. Potongan property test Cauchy-Schwarz:

```
fn toleransi(skala: f64) -> f64 {
    EPS * (1.0 + skala)
}

fn cauchy_schwarz(ax in -1e3f64..1e3, ay in -1e3f64..1e3, bx in -1e3f64..1e3, by in -1e3f64..1e3) {
    let (a, b): (Vector2<f64>, Vector2<f64>) = (Vector2::new(ax, ay), Vector2::new(bx, by));
    let batas = a.norm() * b.norm();
    prop_assert!(a.dot(&b).abs() <= batas + toleransi(batas));
}
```

2.3 Penjelasan Proses

`norma_p` bekerja dalam satu kali jalan. Tiap komponen diambil nilai mutlaknya, dipangkatkan p, dijumlahkan, lalu hasil jumlahnya dipangkatkan 1/p. Cabang `is_infinite()` menangani p tak hingga dengan `fold` yang mencari nilai mutlak terbesar dengan `NEG_INFINITY` sebagai nilai awal.

`lp_norm` milik nalgebra hanya menerima p bertipe i32, jadi norma L-inf di sisi nalgebra dihitung manual dengan `fold` seperti catatan API di Tabel 1 modul. Makro `proptest!` membangkitkan 200 kombinasi komponen di rentang -1000 sampai 1000 untuk tiap sifat. Kalau ada kasus yang gagal, `proptest` mengecilkan input sampai ketemu contoh gagal yang paling sederhana.

2.4 Output Program

Hasil eksekusi `cargo run -- 1` ditampilkan berikut ini.

```
praktikum-al — ghaisan@Ghaisans-MacBook-Pro — ../praktikum-al — zsh — 100x14
> cargo run -- 1
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.05s
    Running `target/debug/praktikum-al 1`
=== Task I: Norma-p Manual & Verifikasi Sifat ===
v = [3.0000, -4.0000]
manual : ||v||_1 = 7, ||v||_2 = 5, ||v||_inf = 4
nalgebra: ||v||_1 = 7, ||v||_2 = 5, ||v||_inf = 4
urutan : 7 >= 5 >= 4 -> true

~/Documents/Kuliah/aljabar-linear/2026-09-14-modul-2/praktikum-al on main ?1 at 09:07:15
> █
```

Gambar 2.1: Hasil cargo run untuk Task I

Norma manual dan nalgebra sama persis: $\|v\|_1 = 7$, $\|v\|_2 = 5$, dan $\|v\|_{\text{inf}} = 4$. Urutannya juga menunjukkan hubungan umum norma-p, yaitu nilainya turun saat p membesar. L1 menjumlahkan semua komponen, L2 memberi bobot lebih pada komponen besar, dan L-inf hanya melihat komponen terbesar.

2.5 Lesson Learnt

Toleransi epsilon ternyata wajib, dan bentuknya harus relatif. Awalnya saya ingin memakai epsilon mutlak $1e-12$ seperti contoh di modul. Masalahnya, untuk vektor sejajar ketaksamaan Cauchy-Schwarz berubah menjadi persamaan, sehingga kedua ruas hanya dipisahkan galat pembulatan floating point. Galat itu ikut membesar mengikuti ukuran vektor. Saya cek dengan skrip terpisah memakai 100.000 pasangan vektor sejajar acak berkomponen sampai 1000: sisi kiri melampaui sisi kanan ditambah $1e-12$ sebanyak 22.324 kali, dengan selisih terbesar $7,45e-9$. Dengan toleransi $1e-12 \times (1 + \text{skala})$, jumlah kegagalannya nol.

Jadi epsilon diperlukan setiap kali membandingkan dua hasil hitung floating point yang secara matematis bisa sama persis, misalnya ruas kiri dan kanan sebuah ketaksamaan atau hasil manual lawan hasil library. Untuk nilai yang memang pasti, seperti norma vektor nol, perbandingan langsung dengan 0.0 tetap aman.

BAB III TASK II: JARAK, KESAMAAN KOSINUS, DAN PROYEKSI

3.1 Deskripsi Program

Task II menghitung jarak Euclidean dan Manhattan, kesamaan kosinus, proyeksi ortogonal, dan normalisasi untuk $u = (1, 2, 3)$ dan $v = (4, 0, -1)$. Kesamaan kosinus didefinisikan $\cos \theta = (u \cdot v) / (\|u\| \|v\|)$, sedangkan proyeksi u ke v adalah $\text{proj}_v u = ((u \cdot v) / \|v\|^2) v$. Sisa $u - \text{proj}_v u$ harus tegak lurus terhadap v . Kasus vektor nol ditangani eksplisit karena sudut dan arahnya tidak terdefinisi.

3.2 Kode Program

```
pub fn cos_sim(u: &Vector3<f64>, v: &Vector3<f64>) -> Option<f64> {
    let d = u.norm() * v.norm();
    if d < EPS { None } else { Some(u.dot(v) / d) }
}

pub fn proyeksi(u: &Vector3<f64>, v: &Vector3<f64>) -> Option<(Vector3<f64>, Vector3<f64>)> {
    let nn = v.norm_squared();
    if nn < EPS {
        return None;
    }
    let proj = (u.dot(v) / nn) * v;
    Some((proj, u - proj))
}
```

3.3 Penjelasan Proses

Jarak dihitung dari selisih dua vektor: $(u - v).norm()$ untuk Euclidean dan $(u - v).lp_norm(1)$ untuk Manhattan. `cos_sim` mengembalikan `Option<f64>`. Kalau hasil kali kedua norma lebih kecil dari $1e-12$, fungsi mengembalikan `None` alih-alih membagi dengan nol. Tanpa pengecekan itu hasilnya NaN, dan NaN bisa diam-diam merusak perhitungan berikutnya karena setiap perbandingan dengan NaN bernilai false.

`proyeksi` memakai pola yang sama. Proyeksi dihitung dari `norm_squared()`, sehingga tidak perlu akar kuadrat, lalu komponen tegak lurusnya didapat dari $u - \text{proj}$. Normalisasi memakai `try_normalize(1e-12)` bawaan nalgebra, yang juga mengembalikan `None` untuk vektor nol.

3.4 Output Program

Hasil eksekusi `cargo run -- 2` ditampilkan berikut ini.

```
praktikum-al — ghaisan@Ghaisans-MacBook-Pro — ../praktikum-al — -zsh — 100x19
> cargo run -- 2
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.02s
    Running `target/debug/praktikum-al 2`
=== Task II: Jarak, Kesamaan Kosinus & Proyeksi ===
u = [1.0000, 2.0000, 3.0000], v = [4.0000, 0.0000, -1.0000]
jarak Euclidean = 5.3852
jarak Manhattan = 9.0000
cos(u, v) = 0.0648, sudut = 86.2835 derajat
cos(u, 0) = None (vektor nol tidak punya arah)
proj_v u = [0.2353, 0.0000, -0.0588]
tegak = [0.7647, 2.0000, 3.0588]
tegak . v = 0e0 (harus nol)
proj+tegak = [1.0000, 2.0000, 3.0000] (kembali ke u)
unit v = [0.9701, 0.0000, -0.2425], ||unit v|| = 1.000000000000
normalisasi vektor nol: None

~/Documents/Kuliah/aljabar-linear/2026-09-14-modul-2/praktikum-al on main ?1 at 09:06:06
>
```

Gambar 3.1: Hasil cargo run untuk Task II

Jarak Euclidean u dan v adalah akar $29 = 5,3852$, sedangkan jarak Manhattan $3 + 2 + 4 = 9$. Kosinusnya hanya $0,0648$, jadi kedua vektor hampir tegak lurus dengan sudut $86,28$ derajat. Karena $u \cdot v = 1$ dan $\|v\|^2 = 17$, proyeksinya $(1/17)v = (0,2353; 0; -0,0588)$, dan hasil kali titik komponen tegak lurus dengan v tercetak $0e0$, artinya ortogonalitasnya terpenuhi sampai bit terakhir. Jumlah proyeksi dan komponen tegak lurus kembali tepat ke u . Untuk vektor nol, kosinus dan normalisasi sama-sama menghasilkan `None`.

3.5 Lesson Learnt

Untuk teks, yang penting adalah arah vektor, bukan panjangnya. Dua dokumen dengan topik sama tetapi panjang berbeda punya vektor frekuensi yang searah, hanya beda skala. Jarak Euclidean menganggap keduanya jauh, sedangkan kosinus menganggapnya mirip, dan inilah yang dibuktikan di Task III.

Kosinus terhadap vektor nol tidak terdefinisi karena vektor nol tidak punya arah. Dokumen kosong tidak mirip dan tidak berbeda dengan dokumen apa pun. Memakai `Option` membuat keadaan itu harus ditangani pemanggil dengan `match`, bukan dibiarkan jadi `NaN` atau dipaksa menjadi `0`.

BAB IV TASK III: STUDI KASUS KEMIRIPAN DOKUMEN

4.1 Deskripsi Program

Kosakata yang dipakai adalah rust, vektor, matriks, cepat, dan aman. Tiga dokumen dan satu kueri dikodekan sebagai `SVector<f64, 5>` berisi frekuensi kata (TF): d1 dokumen Rust (3, 2, 1, 1, 2), d2 dokumen teori (1, 3, 2, 0, 1), d3 dokumen iklan (4, 0, 0, 3, 3), dan kueri (2, 1, 0, 1, 1). Program membuat peringkat dengan kosinus dan dengan jarak Euclidean, lalu mengulanginya setelah d3 dikali 100 untuk mensimulasikan dokumen yang sangat panjang.

4.2 Kode Program

```
pub fn peringkat_kosinus(q: &Dok, docs: &[(&'static str, Dok)]) -> Vec<(&'static str, f64)> {
    let mut hasil: Vec<_> = docs
        .iter()
        .map(|(nama, d)| (*nama, cos_sim(q, d).unwrap_or(0.0)))
        .collect();
    hasil.sort_by(|a, b| b.1.total_cmp(&a.1));
    hasil
}

pub fn peringkat_euclidean(q: &Dok, docs: &[(&'static str, Dok)]) -> Vec<(&'static str, f64)> {
    let mut hasil: Vec<_> = docs
        .iter()
        .map(|(nama, d)| (*nama, (q - d).norm()))
        .collect();
    hasil.sort_by(|a, b| a.1.total_cmp(&b.1));
    hasil
}
```

4.3 Penjelasan Proses

Kedua fungsi peringkat memetakan tiap dokumen menjadi pasangan nama dan skor, lalu mengurutkannya dengan `total_cmp`, yaitu perbandingan `f64` yang tetap punya urutan pasti walaupun ada `NaN`. Kosinus diurutkan menurun karena makin besar makin mirip, sedangkan jarak diurutkan menaik karena makin kecil makin dekat. Eksperimen skala memakai `docs[2].1 *= 100.0`, sehingga isi d3 tetap sama tetapi setiap kata seolah muncul 100 kali lebih sering.

4.4 Output Program

Hasil eksekusi `cargo run -- 3` ditampilkan berikut ini.

```

praktikum-al — ghaisan@Ghaisans-MacBook-Pro — ../praktikum-al — zsh — 100x40

> cargo run -- 3
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.03s
    Running `target/debug/praktikum-al 3`
=== Task III: Studi Kasus Kemiripan Dokumen ===
kosakata = ["rust", "vektor", "matriks", "cepat", "aman"]
kueri     = [2.0000, 1.0000, 0.0000, 1.0000, 1.0000]
d1 (dokumen Rust) = [3.0000, 2.0000, 1.0000, 1.0000, 2.0000]
d2 (dokumen teori) = [1.0000, 3.0000, 2.0000, 0.0000, 1.0000]
d3 (dokumen iklan) = [4.0000, 0.0000, 0.0000, 3.0000, 3.0000]

Peringkat kesamaan kosinus (besar = mirip):
1. d1 (dokumen Rust)      0.9538
2. d3 (dokumen iklan)     0.9075
3. d2 (dokumen teori)     0.5855

Peringkat jarak Euclidean (kecil = mirip):
1. d1 (dokumen Rust)      2.0000
2. d2 (dokumen teori)     3.1623
3. d3 (dokumen iklan)     3.6056

Setelah d3 dikali 100 (dokumen panjang):
Peringkat kesamaan kosinus:
1. d1 (dokumen Rust)      0.9538
2. d3 (dokumen iklan)     0.9075
3. d2 (dokumen teori)     0.5855
Peringkat jarak Euclidean:
1. d1 (dokumen Rust)      2.0000
2. d2 (dokumen teori)     3.1623
3. d3 (dokumen iklan)     580.6953

~/Documents/Kuliah/aljabar-linear/2026-09-14-modul-2/praktikum-al on main ?1 at 09:07:23
>

```

Gambar 4.1: Hasil cargo run untuk Task III

Tabel 4.1: Perbandingan skor kueri terhadap ketiga dokumen

Dokumen	Kosinus	Jarak Euclidean	Setelah d3 x 100
d1 (dokumen Rust)	0,9538 (1)	2,0000 (1)	tidak berubah
d2 (dokumen teori)	0,5855 (3)	3,1623 (2)	tidak berubah
d3 (dokumen iklan)	0,9075 (2)	3,6056 (3)	kosinus 0,9075, jarak 580,6953

4.5 Analisis dan Lesson Learnt

Kedua ukuran sepakat bahwa d1 paling mirip dengan kueri, tetapi urutan kedua dan ketiganya berbeda. Kosinus menaruh d3 di posisi kedua. Kueri berbobot pada kata rust, cepat, dan aman, dan d3 juga hanya berisi tiga kata itu, jadi arahnya hampir sama walaupun jumlah katanya jauh lebih banyak. Jarak Euclidean justru menaruh d3 terakhir karena selisih besarnya frekuensi ikut dihitung sebagai jarak. d2 yang topiknya berbeda (vektor dan matriks) malah naik ke posisi kedua hanya karena ukurannya lebih dekat ke kueri.

Eksperimen skala membuat perbedaannya jelas. Setelah d_3 dikali 100, skor kosinusnya tetap 0,9075 sampai digit terakhir, sedangkan jarak Euclidean melonjak dari 3,6056 menjadi 580,6953. Kosinus invarian terhadap skala karena faktor pengali muncul di pembilang dan penyebut lalu saling menghapus. Norma tidak punya sifat itu, sehingga Euclidean diam-diam menghukum dokumen panjang. Untuk mesin pencari, dokumen panjang yang relevan tidak boleh kalah hanya karena panjang, jadi kosinus adalah pilihan yang lebih tepat untuk data frekuensi kata.

BAB V TUGAS DAN LATIHAN

5.1 Latihan 1: Hubungan Norma

Latihan 1 membuktikan secara numerik $\|v\|_2 \leq \|v\|_1 \leq \sqrt{n} \|v\|_2$ untuk 100 vektor acak dengan $n = 8$. Supaya tidak menambah crate `rand`, angka acaknya dibangkitkan `xorshift64` sederhana dengan seed tetap, jadi hasilnya bisa diulang. Seluruh 100 vektor memenuhi, dan rasio $\|v\|_1 / (\sqrt{8} \|v\|_2)$ terbesar hanya 0,9752. Rasio itu baru mencapai 1 kalau semua komponen bernilai mutlak sama. Sifat yang sama juga diuji ulang dengan `proptest`.

5.2 Latihan 2: Proyeksi dan Sudut

Untuk $u = (2, 1, -2)$ dan $v = (1, -1, 1)$, hitungan tangannya $u \cdot v = 2 - 1 - 2 = -1$ dan $\|v\|^2 = 3$. Proyeksinya $\text{proj}_v u = (-1/3) v = (-1/3, 1/3, -1/3)$, dan komponen tegak lurusnya $u - \text{proj}_v u = (7/3, 2/3, -5/3)$. Hasil kali titiknya dengan v adalah $7/3 - 2/3 - 5/3 = 0$. Karena $\|u\| = 3$ dan $\|v\| = \sqrt{3}$, $\cos \theta = -1 / (3 \sqrt{3}) = -0,1925$, sehingga sudutnya 101,0958 derajat. Semua angka ini sama dengan output program pada Gambar 5.1.

5.3 Tugas Rumah: Jarak Minkowski

```
pub fn jarak_minkowski<const D: usize>(
    u: &SVector<f64, D>,
    v: &SVector<f64, D>,
    p: f64,
) -> Option<f64> {
    if p.is_nan() || p < 1.0 {
        return None;
    }
    Some(norma_p((u - v).as_slice(), p))
}
```

Fungsi dibuat generik terhadap dimensi dengan `const generic D`, jadi satu fungsi berlaku untuk `Vector3`, `SVector<f64, 5>`, dan dimensi lain. Nilai p kurang dari 1 atau NaN ditolak dengan `None`, karena untuk $p < 1$ ketaksamaan segitiga tidak berlaku lagi sehingga hasilnya bukan jarak. Unit test mencakup $p = 1, 2$, dan tak hingga (7, 5, dan 4 untuk $u = (1, 2, 3)$ dan $v = (4, 6, 3)$), kesamaan dengan nalgebra pada vektor 4 dimensi, jarak ke diri sendiri bernilai nol, p tidak valid, dan sifat simetris lewat `proptest`.

Tabel 5.1: Analisis kompleksitas jarak Minkowski untuk vektor berdimensi n

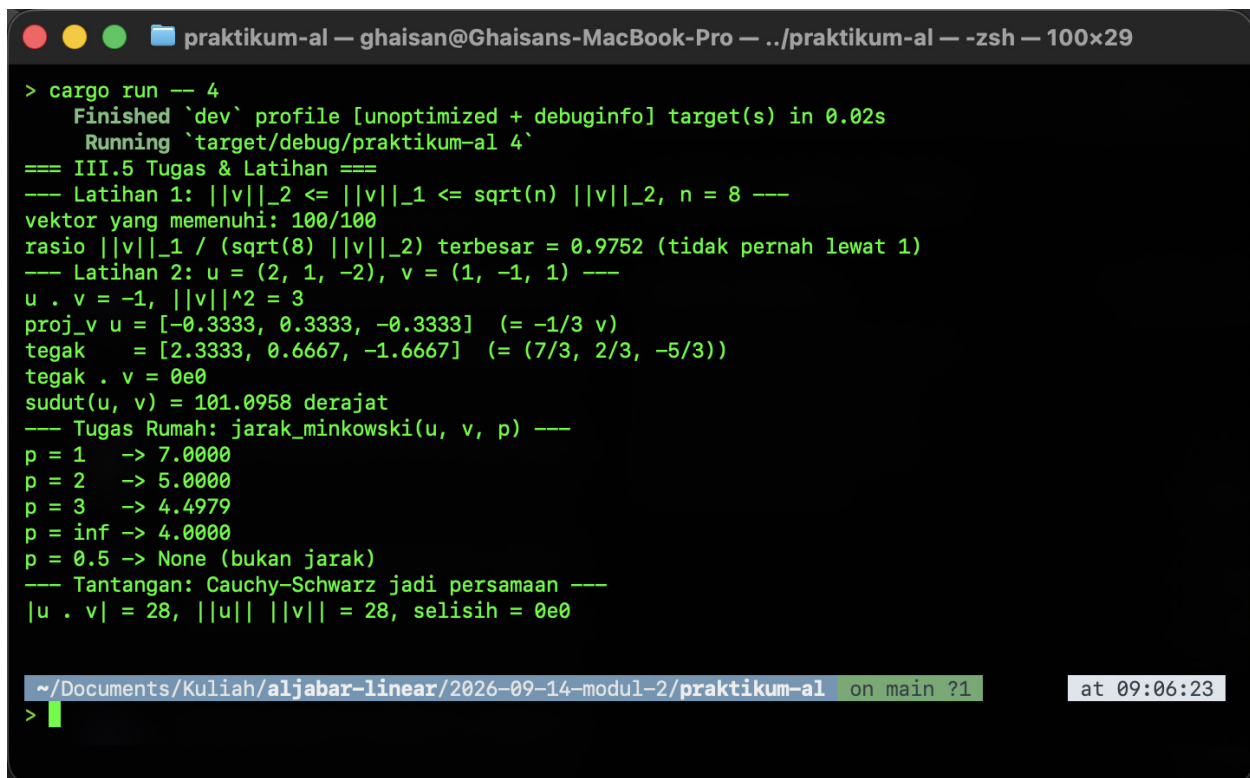
Varian	Operasi per komponen	Waktu	Memori
$p = 1$ (Manhattan)	nilai mutlak lalu dijumlah	$O(n)$	$O(n)$
$p = 2$ (Euclidean)	nilai mutlak, pangkat 2, dijumlah, satu akar di akhir	$O(n)$	$O(n)$
p umum	$\text{powf}(p)$ per komponen, satu $\text{powf}(1/p)$ di akhir	$O(n)$	$O(n)$
$p = \text{tak hingga}$	nilai mutlak lalu dicari maksimum dengan fold	$O(n)$	$O(n)$

Semua varian $O(n)$ karena setiap komponen dikunjungi tepat sekali dan operasi per komponen berbiaya konstan. Perbedaan hanya di konstanta: `powf` jauh lebih mahal daripada `abs`, sehingga `p` umum paling lambat walaupun notasi O -nya sama. Memori $O(n)$ berasal dari vektor selisih $u - v$. Kalau selisih dihitung langsung saat iterasi dengan `zip`, memorinya bisa turun menjadi $O(1)$.

5.4 Tantangan: Cauchy-Schwarz Menjadi Persamaan

Ketaksamaan $|u \cdot v| \leq \|u\| \|v\|$ menjadi persamaan tepat ketika kedua vektor sejajar. Dengan $u = (1, 2, 3)$ dan $v = 2u$, $|u \cdot v| = 28$ dan $\|u\| \|v\| = \text{akar } 14 \times 2 \text{ akar } 14 = 28$, selisihnya tercetak `0e0`. Secara geometris sudutnya 0 derajat sehingga $\cos \theta = 1$, dan tidak ada komponen v yang tegak lurus terhadap u .

Hasil eksekusi `cargo run -- 4` untuk seluruh latihan ditampilkan berikut ini.



```
praktikum-al — ghaisan@Ghaisans-MacBook-Pro — ../praktikum-al — zsh — 100x29
> cargo run -- 4
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.02s
  Running `target/debug/praktikum-al 4`
=== III.5 Tugas & Latihan ===
--- Latihan 1:  $\|v\|_2 \leq \|v\|_1 \leq \sqrt{n} \|v\|_2$ ,  $n = 8$  ---
vektor yang memenuhi: 100/100
rasio  $\|v\|_1 / (\sqrt{8} \|v\|_2)$  terbesar = 0.9752 (tidak pernah lewat 1)
--- Latihan 2:  $u = (2, 1, -2)$ ,  $v = (1, -1, 1)$  ---
 $u \cdot v = -1$ ,  $\|v\|^2 = 3$ 
proj_v u = [-0.3333, 0.3333, -0.3333] (= -1/3 v)
tegak    = [2.3333, 0.6667, -1.6667] (= (7/3, 2/3, -5/3))
tegak . v = 0e0
sudut(u, v) = 101.0958 derajat
--- Tugas Rumah: jarak_minkowski(u, v, p) ---
p = 1   -> 7.0000
p = 2   -> 5.0000
p = 3   -> 4.4979
p = inf -> 4.0000
p = 0.5 -> None (bukan jarak)
--- Tantangan: Cauchy-Schwarz jadi persamaan ---
 $|u \cdot v| = 28$ ,  $\|u\| \|v\| = 28$ , selisih = 0e0

~/Documents/Kuliah/aljabar-linear/2026-09-14-modul-2/praktikum-al on main ?1 at 09:06:23
>
```

Gambar 5.1: Hasil cargo run untuk latihan, tugas rumah, dan tantangan

BAB VI PENGUJIAN OTOMATIS DAN KUALITAS KODE

6.1 Unit Test dan Property Test

Total ada 28 test. 18 di antaranya unit test dengan nilai yang bisa dihitung tangan, dan 10 property test proptest yang masing-masing mencoba 100 sampai 200 input acak.

Tabel 6.1: Ringkasan pengujian per berkas

Berkas	Unit	Property	Yang diuji
task1.rs	3	5	norma contoh modul, kesamaan dengan nalgebra, norma vektor nol, homogenitas (k acak dan $k = 2$), segitiga, Cauchy-Schwarz, urutan $L1 \geq L2 \geq L\text{-inf}$
task2.rs	5	2	jarak contoh, kosinus searah, tegak lurus, dan berlawanan, vektor nol, normalisasi, ortogonalitas proyeksi acak, rentang kosinus $[-1, 1]$
task3.rs	4	1	urutan peringkat kosinus dan Euclidean, nilai kosinus d1, eksperimen skala 100, invariansi skala positif
latihan.rs	6	2	Minkowski $p = 1, 2$, tak hingga, kesamaan nalgebra, p tidak valid, jarak ke diri sendiri, Latihan 1, Latihan 2, tantangan, simetri

6.2 Hasil Pengujian

Hasil eksekusi `cargo test` ditampilkan berikut ini.

```
praktikum-al — ghaisan@Ghaisans-MacBook-Pro — ../praktikum-al — zsh — 100x40

> cargo test
  Finished `test` profile [unoptimized + debuginfo] target(s) in 0.04s
  Running unittests src/main.rs (target/debug/deps/praktikum_al-bcba3588647da0d4)

running 28 tests
test latihan::tests::minkowski_p_tidak_valid ... ok
test latihan::tests::minkowski_p1_p2_pinf ... ok
test latihan::tests::latihan2_proyeksi ... ok
test latihan::tests::tantangan_cauchy_schwarz_sejajar ... ok
test latihan::tests::minkowski_ke_diri_sendiri_nol ... ok
test latihan::tests::minkowski_sama_dengan_nalgebra ... ok
test task1::tests::manual_sama_dengan_nalgebra ... ok
test task1::tests::norma_p_contoh_modul ... ok
test task1::tests::norma_vektor_nol_adalah_nol ... ok
test task2::tests::jarak_contoh ... ok
test task2::tests::kosinus_searah_tegak_lurus_berlawanan ... ok
test task2::tests::kosinus_vektor_nol_none ... ok
test task2::tests::normalisasi ... ok
test task2::tests::proyeksi_ke_vektor_nol_none ... ok
test latihan::tests::minkowski_simetris ... ok
test task3::tests::nilai_kosinus_d1 ... ok
test task3::tests::skala_100_kosinus_tetap_euclidean_berubah ... ok
test task3::tests::urutan_euclidean_d1_d2_d3 ... ok
test task3::tests::urutan_kosinus_d1_d3_d2 ... ok
test task1::tests::homogenitas_kali_dua ... ok
test task1::tests::homogenitas ... ok
test task1::tests::cauchy_schwarz ... ok
test task1::tests::segitiga ... ok
test task2::tests::kosinus_selalu_di_rentang ... ok
test latihan::tests::latihan1_hubungan_norma ... ok
test task1::tests::norma_p_turun_saat_p_naik ... ok
test task3::tests::kosinus_invarian_skala_positif ... ok
test task2::tests::komponen_tegak_ortogonal ... ok

test result: ok. 28 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.01s

~/Documents/Kuliah/aljabar-linear/2026-09-14-modul-2/praktikum-al on main ?1 at 09:06:31
> 
```

Gambar 6.1: Hasil cargo test, 28 test lolos

Seluruh 28 test lolos dalam 0,01 detik. Riwayat lengkapnya juga disimpan sebagai berkas teks riwayat-cargo-test.txt di dalam arsip pengumpulan.

6.3 Kualitas Kode

cargo build selesai tanpa warning, cargo clippy --all-targets -- -D warnings bersih seperti pada Gambar 1.1, dan seluruh berkas sudah dirapikan dengan cargo fmt. Satu galat yang sempat muncul sama dengan jebakan di Modul 1: `Vector3::new(2.0, 1.0, -2.0)` tanpa anotasi tipe membuat pemanggilan `.acos()` dan `.abs()` ditolak dengan E0282 dan E0689, karena kompilator belum tahu literal itu `f64`. Solusinya menulis `let u: Vector3<f64>` secara eksplisit.

BAB VII KESIMPULAN

1. Norma-p manual memberi hasil yang sama dengan nalgebra, yaitu 7, 5, dan 4 untuk $v = [3, -4]$. Sifat homogenitas, ketaksamaan segitiga, dan Cauchy-Schwarz lolos pada ratusan vektor acak.
2. Pengujian numerik perlu toleransi yang ikut membesar mengikuti ukuran nilai. Epsilon mutlak $1e-12$ gagal pada 22.324 dari 100.000 pasangan vektor sejajar, sedangkan toleransi relatif tidak pernah gagal.
3. Kesamaan kosinus dan proyeksi harus menangani vektor nol secara eksplisit. `Option` membuat kasus itu tidak bisa terlewat, dan ortogonalitas komponen tegak lurus terbukti lewat test.
4. Pada studi kasus dokumen, kosinus dan Euclidean memberi peringkat berbeda. Kosinus tetap 0,9075 saat d3 dikali 100, sedangkan jarak Euclidean melonjak ke 580,6953, sehingga kosinus lebih tepat untuk data frekuensi kata.
5. Jarak Minkowski generik berjalan $O(n)$ untuk semua nilai p , dengan memori tambahan $O(n)$ untuk vektor selisih.